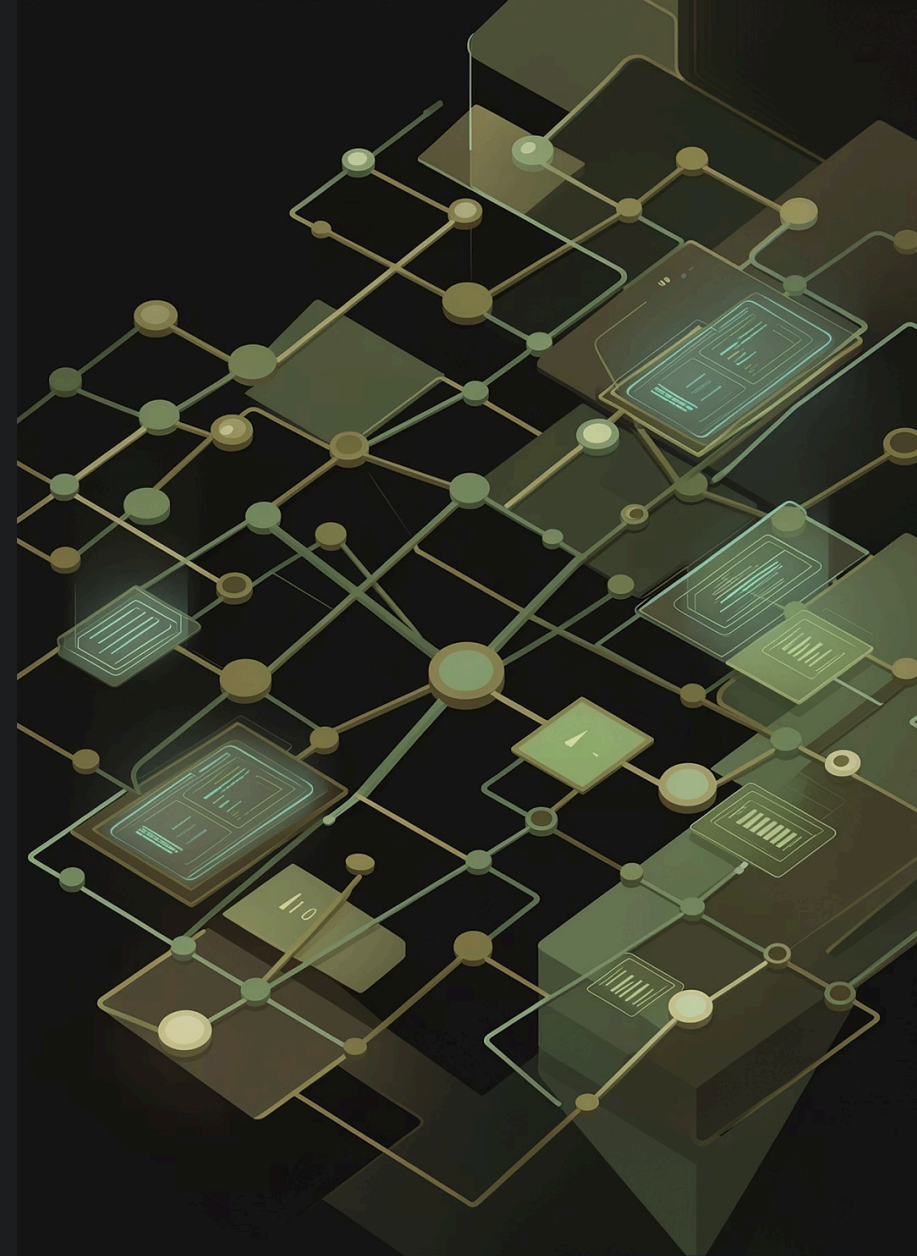


Análise de Eficiência Computacional: Vetorização vs. Processamento Iterativo em Regressão Logística

Estudo de caso com Heart Disease Dataset - Grupo 2

Discentes: Eduardo Lucena, Francisco Gutierrys, Gabriel Siqueira

Disciplina: Estrutura de Dados II | Professor: Adam D. F. dos Santos



Introdução e Motivação

Contexto

A evolução do Machine Learning demanda processamento eficiente de volumes massivos de dados para extração de padrões.

Desafio

O custo computacional de laços *loops* em Python para grandes matrizes representa um gargalo crítico.

"A extração de padrões define o sucesso da IA moderna"

— Goodfellow et al., 2016

Fundamentação Teórica



Regressão Logística

Modelo de classificação binária baseado na função Sigmoide para mapeamento probabilístico.



Gradiente Descendente

Algoritmo iterativo para minimização da função de custo através de atualização de parâmetros.



Vetorização

Substituição de laços por operações matriciais utilizando rotinas otimizadas BLAS/LAPACK/SIMD.

Metodologia e Dataset

Heart Disease Dataset

- 1025 amostras para análise
- 13 categorias trabalhadas
- Fonte: Kaggle

Ambiente

- Google Colab
- Python 3.12
- CPU via Google Compute Engine

Pré-processamento de Dados

- Limpeza de valores nulos
- Label Encoding para variáveis categóricas
- Remoção de categoria desnecessária
- Padronização via StandardScaler

Banco de Dados (Tabela Inicial)

Using Colab cache for faster access to the 'heart-disease-dataset' dataset.

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

Pré-Processamento de Dados

1. Verificação de valores ausentes

```
# PRE-PROCESSAMENTO DE DADOS

# Verificar e tratar valores ausentes
missing_count = df.isnull().sum()
print('Valores ausentes por coluna:')
print(missing_count[missing_count > 0] if missing_count.sum() > 0 else 'Nenhum valor ausente encontrado.')

df = df.dropna()
print('\nTratamento concluído. Dimensões atuais do DataFrame:', df.shape)
```

Valores ausentes por coluna:
Nenhum valor ausente encontrado.

Tratamento concluído. Dimensões atuais do DataFrame: (1025, 14)

2. Codificação e Remoção do Target

```
# PRE-PROCESSAMENTO DE DADOS

# Codificação de variáveis categóricas
# Metodo Label Encoding utilizado

# Identificando colunas categoricas
categorical_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']

# Aplicando Label Encoding
from sklearn.preprocessing import LabelEncoder
df_encoded = df.copy() # Criar uma cópia para não modificar o DataFrame original
for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])

print('Variáveis codificadas com sucesso usando Label Encoding.')
display(df_encoded.drop('target', axis=1).head())
```

... Variáveis codificadas com sucesso usando Label Encoding.

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

3. Padronização

```
# PRE-PROCESSAMENTO DE DADOS

# import do StandardScaler para padronizar os dados
from sklearn.preprocessing import StandardScaler

# Identificando colunas numericas (não codificadas)
numeric_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

# Inicializando o escalonador
scaler = StandardScaler()

# Criando uma copia para preservar os dados originais e aplicando a padronização
df_scaled = df_encoded.copy()
df_scaled[numeric_cols] = scaler.fit_transform(df_encoded[numeric_cols])

print('Padronização concluída com sucesso.')
display(df_scaled.drop('target', axis=1).head())
```

Padronização concluída com sucesso.

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	-0.268437	1	0	-0.377636	-0.659332	0	1	0.821321	0	-0.060888	2	2	3
1	-0.158157	1	0	0.479107	-0.833861	1	0	0.255968	1	1.727137	0	0	3
2	1.716595	1	0	0.764688	-1.396233	0	1	-1.048692	1	1.301417	0	0	3
3	0.724079	1	0	0.936037	-0.833861	0	1	0.516900	0	-0.912329	2	1	3
4	0.834359	0	0	0.364875	0.930822	1	1	-1.874977	0	0.705408	1	3	2

4. Divisão do Conjunto de Treinamento e Teste

```
import numpy as np
# PRE-PROCESSAMENTO DE DADOS

# import do train_test_split para dividir o conjunto de dados em dois
from sklearn.model_selection import train_test_split

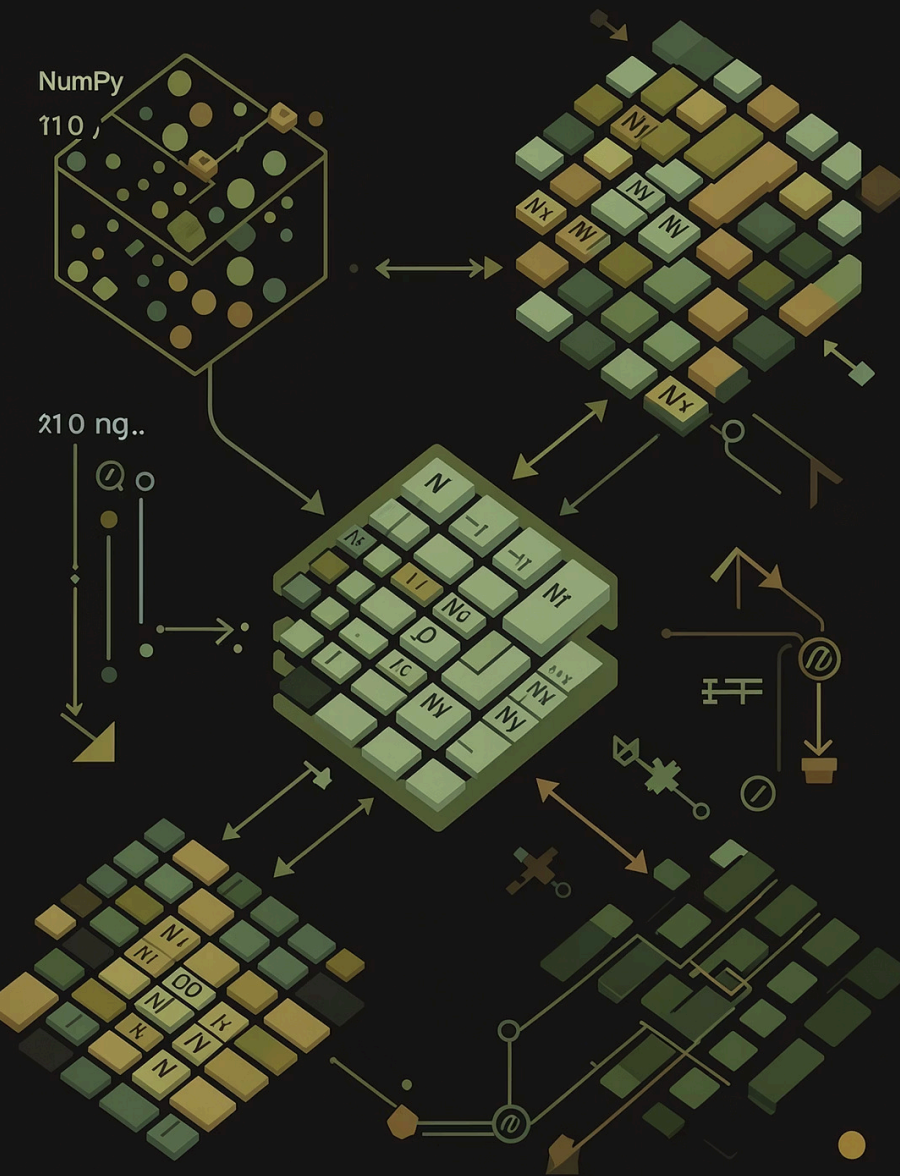
# Separando as features (X) e o alvo (y)
X = df_scaled.drop('target', axis=1)
y = df_scaled['target']

# Dividindo em treino (80%) e teste (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f'Divisão concluída:')
print(f'X_train: {X_train.shape}')
print(f'X_test: {X_test.shape}')

# Converter para numpy (necessário para implementação não-vetorizada)
X_train = X_train.values
X_test = X_test.values
y_train = y_train.values
y_test = y_test.values
```

```
... Divisão concluída:
X_train: (820, 13)
X_test: (205, 13)
```



Implementação Vetorizada (NumPy)

1

Estrutura

Operações matriciais processando todos os exemplos simultaneamente via NumPy.

2

Comportamento

Execução otimizada em nível de hardware com instruções paralelas.

3

Tempo

0,0286 segundos em média por execução.

Implementação Vetorizada

```
#importacao do numpy para implementacao vetorizada
import numpy as np

#funcao sigmoide
def sigmoid(z):
    return 1 / (1 + np.exp(-z.astype(float)))

# Preparacao dos dados: X deve ter shape (n, m) onde n é o numero de features e m o numero de exemplos
# Nossas features originais estao em X_train (m, n), então transpomos.
X_vec_train = X_train.T.astype(float)
y_vec_train = y_train.reshape(1, -1).astype(float)

X_vec_test = X_test.T.astype(float)
y_vec_test = y_test.reshape(1, -1).astype(float)

# Inicialização de parâmetros: w como (n, 1) e b como escalar
n_features = X_vec_train.shape[0]
w = np.zeros((n_features, 1))
b = 0.0

learning_rate = 0.1
iterations = 1000
m = X_vec_train.shape[1]
```

```

# Loop de treinamento (Gradiente Descendente Vetorizado)
for i in range(iterations):
    # Forward Propagation:  $Z = w.T \cdot X + b$ 
    Z = np.dot(w.T, X_vec_train) + b
    A = sigmoid(Z)

    # Função de Custo (Log Loss)
    cost = (-1/m) * np.sum(y_vec_train * np.log(A + 1e-15) + (1 - y_vec_train) * np.log(1 - A + 1e-15))

    # Backward Propagation (Gradiente Vetorizado)
    dZ = A - y_vec_train
    dw = (1/m) * np.dot(X_vec_train, dZ.T)
    db = (1/m) * np.sum(dZ)

    # Atualização de Parâmetros
    w -= learning_rate * dw
    b -= learning_rate * db

# Avaliação no conjunto de teste
Z_test = np.dot(w.T, X_vec_test) + b
A_test = sigmoid(Z_test)
y_prediction = (A_test >= 0.5).astype(int)

accuracy = np.mean(y_prediction == y_vec_test)
print(f'Custo final: {cost}')
print(f'Acurácia no Teste: {accuracy * 100:.2f}%')

```

```

... Custo final: 0.33817661324036175
Acurácia no Teste: 79.51%

```



```
f/
tf
python for loops"(pp)}": {
}
mon
aypp sbn soak,
sennher bids; {
"python"
file.ulo();
Pupor bon {}
rh/suion">{
}
}
}
```

Implementação Não-Vetorizada

Estrutura

1

Laços *for* aninhados para cálculo do gradiente e atualização de pesos.

Comportamento

2

Alto custo de interpretação linha a linha pelo interpretador Python.

Tempo

3

11,7155 segundos em média por execução.

1. Implementação Não-Vetorizada

```
# =====  
# IMPLEMENTAÇÃO NÃO-VETORIZADA  
# =====  
  
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))  
  
def train_non_vectorized(X, y, learning_rate=0.1, epochs=1000):  
    m, n = X.shape  
  
    w = np.zeros(n)  
    b = 0  
    cost_nv = []  
  
    for epoch in range(epochs):  
        dw = np.zeros(n)  
        db = 0  
        cost = 0  
  
        for i in range(m):  
            z = 0  
  
            for j in range(n):  
                z += w[j] * X[i][j]  
  
            z += b  
  
            a = sigmoid(z)  
            # Alteração 2: Unificar o tratamento de estabilidade numérica na função de custo  
            # Removido: a = np.clip(a, 1e-15, 1 - 1e-15)  
            cost += -(y[i]*np.log(a + 1e-15) + (1-y[i])*np.log(1 - a + 1e-15))  
  
            dz = a - y[i]  
  
            for j in range(n):  
                dw[j] += X[i][j] * dz  
  
            db += dz  
  
        cost /= m  
        dw /= m  
        db /= m  
  
        for j in range(n):  
            w[j] -= learning_rate * dw[j]  
  
        b -= learning_rate * db  
        cost_nv.append(cost)  
  
    return w, b, cost_nv
```


2. Predição

```
def predict_non_vectorized(X, w, b):  
    m = X.shape[0]  
    y_pred = np.zeros(m)  
  
    for i in range(m):  
        z = 0  
  
        for j in range(len(w)):  
            z += w[j] * X[i][j]  
  
        z += b  
  
        a = sigmoid(z)  
        y_pred[i] = 1 if a >= 0.5 else 0  
  
    return y_pred
```

3. Treinamento e Avaliação

```
# =====  
# TREINAMENTO E AVALIAÇÃO  
# =====  
  
w_nv, b_nv, cost_nv = train_non_vectorized(X_train, y_train)  
  
y_pred_nv = predict_non_vectorized(X_test, w_nv, b_nv)  
  
accuracy_nv = np.mean(y_pred_nv == y_test)  
  
print("Acurácia (não vetorizado):", accuracy_nv)  
print("Custo final:", cost_nv[-1])
```

```
... Acurácia (não vetorizado): 0.7951219512195122  
Custo final: 0.3381766132403616
```



Medição de Tempo

Implementação dos algoritmos de treinamento com inicialização idêntica de parâmetros, utilizando 1000 iterações e taxa de aprendizado fixada em 0,1, repetindo 5 vezes em cada implementação. O objetivo é isolar o impacto dos laços de repetição explícitos frente às operações otimizadas do NumPy.

1. Implementação Vetorizada

```
# =====  
# IMPLEMENTAÇÃO VETORIZADA  
# =====  
print(f"\nIniciando 5 execuções da VETORIZADA")  
tempos_v = []  
n_features = X_vec_train.shape[0]  
m_v = X_vec_train.shape[1]  
  
for rep in range(num_repeticoes):  
    w_v = np.zeros((n_features, 1))  
    b_v = 0.0  
  
    t_inicio = time.perf_counter()  
    for i in range(iterations):  
        Z = np.dot(w_v.T, X_vec_train) + b_v  
        A = sigmoid(Z)  
        dZ = A - y_vec_train  
        dw = (1/m_v) * np.dot(X_vec_train, dZ.T)  
        db = (1/m_v) * np.sum(dZ)  
        w_v -= learning_rate * dw  
        b_v -= learning_rate * db  
  
    t_fim = time.perf_counter()  
    tempos_v.append(t_fim - t_inicio)  
    print(f" V   Repetição {rep+1}: {tempos_v[-1]:.4f} s")
```

2. Implementação Não-Vetorizada

```
# =====  
# IMPLEMENTAÇÃO NÃO-VETORIZADA  
# =====  
print(f"Iniciando 5 execuções da NÃO VETORIZADA")  
tempos_nv = []  
m, n = X_train.shape  
  
for rep in range(num_repeticoes):  
    w_nv = np.zeros(n)  
    b_nv = 0.0  
  
    t_inicio = time.perf_counter()  
    for epoch in range(iterations):  
        dw = np.zeros(n)  
        db = 0.0  
        # Laços explícitos sobre amostras e atributos  
        for i in range(m):  
            z = 0.0  
            for j in range(n):  
                z += w_nv[j] * X_train[i][j]  
            z += b_nv  
            a = 1 / (1 + np.exp(-z))  
            dz = a - y_train[i]  
  
            for j in range(n):  
                dw[j] += X_train[i][j] * dz  
            db += dz  
  
        w_nv -= learning_rate * (dw / m)  
        b_nv -= learning_rate * (db / m)  
  
    t_fim = time.perf_counter()  
    tempos_nv.append(t_fim - t_inicio)  
    print(f" NV Repetição {rep+1}: {tempos_nv[-1]:.4f} s")
```


Comparação de Resultados

1. Execuções

Iniciando 5 execuções da VETORIZADA

V Repetição 1: 0.0287 s

V Repetição 2: 0.0281 s

V Repetição 3: 0.0283 s

V Repetição 4: 0.0282 s

V Repetição 5: 0.0297 s

Iniciando 5 execuções da NÃO VETORIZADA

NV Repetição 1: 11.7801 s

NV Repetição 2: 11.6673 s

NV Repetição 3: 11.7567 s

NV Repetição 4: 11.7083 s

NV Repetição 5: 11.6652 s

=====

TABELA COMPARATIVA

=====

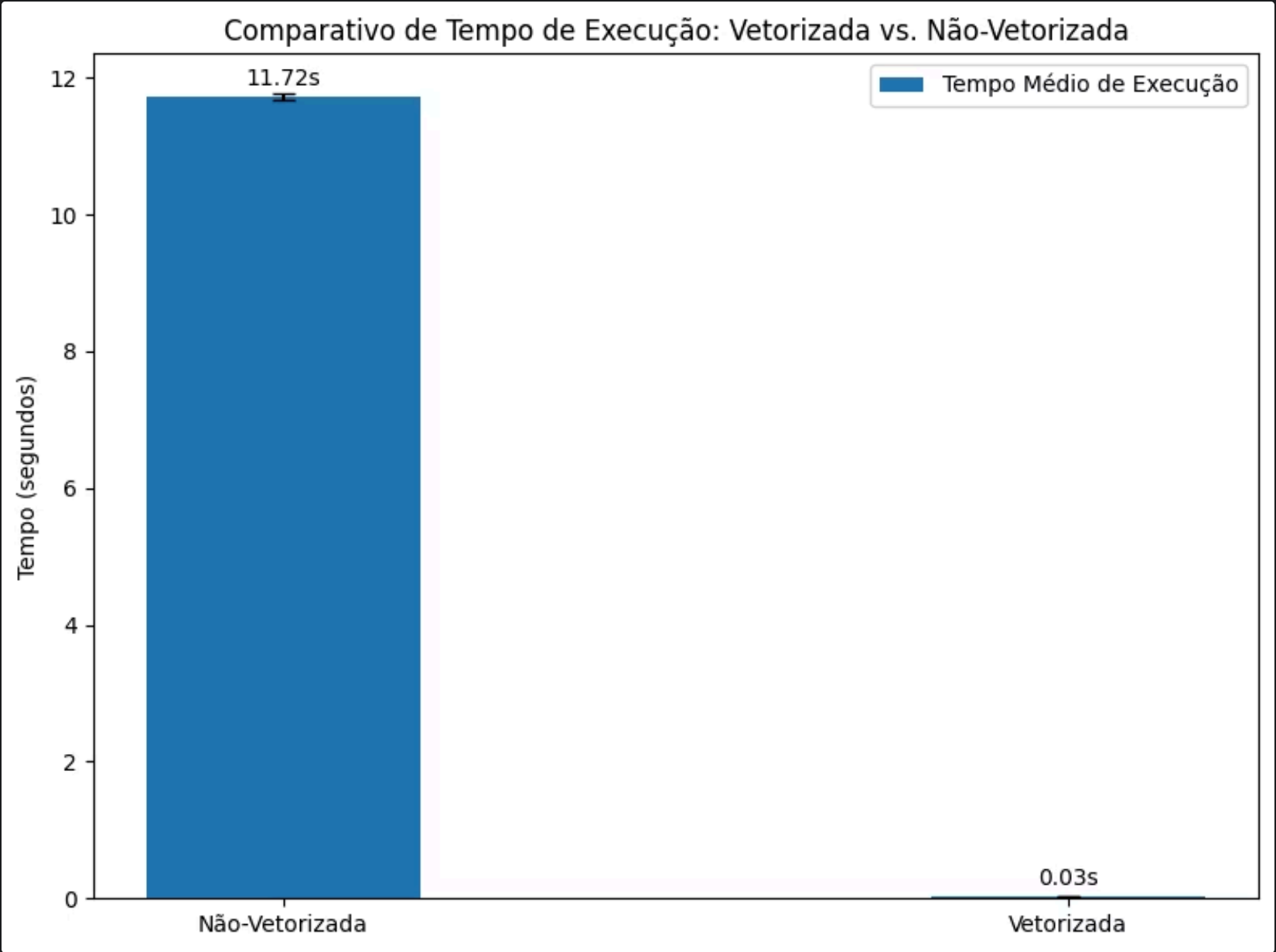
MEDIA NÃO-VETORIZADA: 11.7155 s $\pm$ 0.0464

MEDIA VETORIZADA: 0.0286 s $\pm$ 0.0006

SPEEDUP: 409.99x mais rápido

=====

2. Representação Gráfica do Tempo de Execução



4. Speedup (Após recompilação da medição de tempo com amostras crescentes)

Iniciando experimento para variar o número de amostras (m)...

Testando com m = 100 amostras...

Média de tempo vetorizado: 0.0156 s

Média de tempo não-vetorizado: 1.1264 s

Speedup para m=100: 72.14x

Testando com m = 331 amostras...

Média de tempo vetorizado: 0.0178 s

Média de tempo não-vetorizado: 3.9295 s

Speedup para m=331: 220.52x

Testando com m = 562 amostras...

Média de tempo vetorizado: 0.0224 s

Média de tempo não-vetorizado: 6.7568 s

Speedup para m=562: 301.61x

Testando com m = 793 amostras...

Média de tempo vetorizado: 0.0247 s

Média de tempo não-vetorizado: 9.4248 s

Speedup para m=793: 382.27x

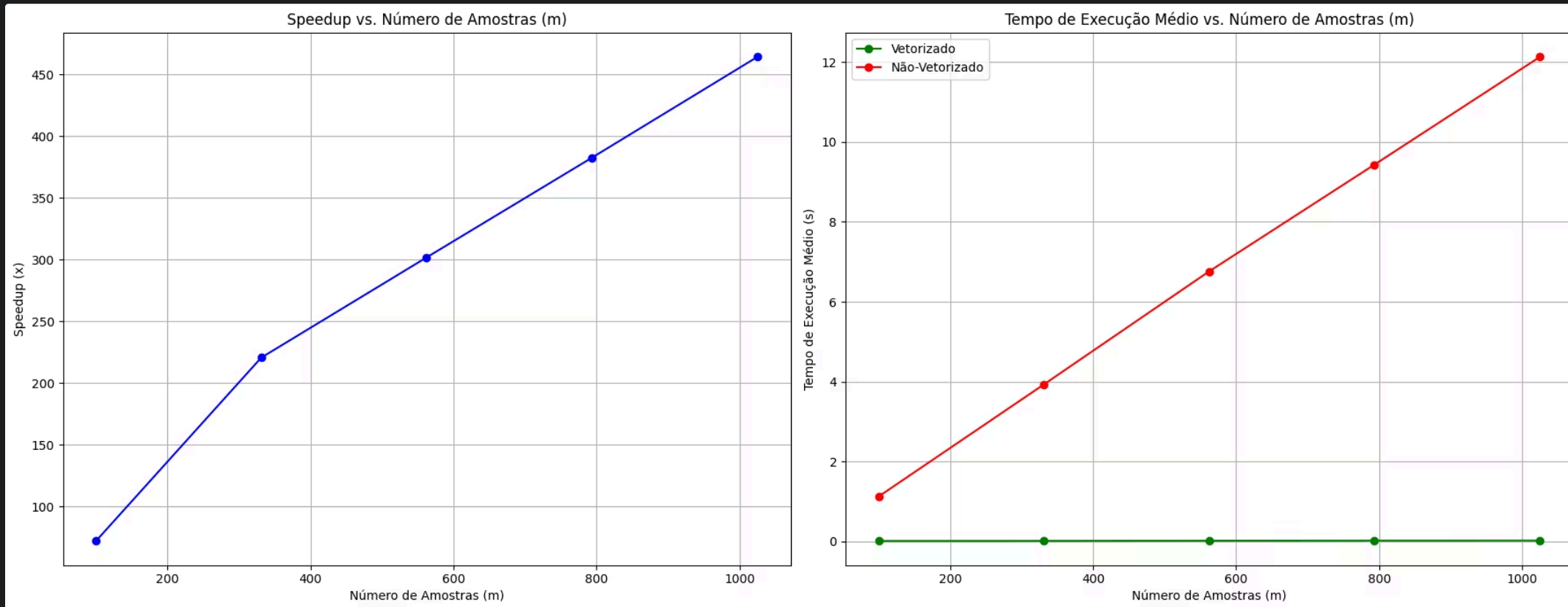
Testando com m = 1025 amostras...

Média de tempo vetorizado: 0.0261 s

Média de tempo não-vetorizado: 12.1271 s

Speedup para m=1025: 464.11x

4. Representação Gráfica do speedup



1. Tabela Comparativa

Versão	Tempo Médio (s)	Desvio Padrão (s)
Não-Vetorizada	11,71	± 0,0464
Vetorizada	0,028	± 0,0006

409.99x

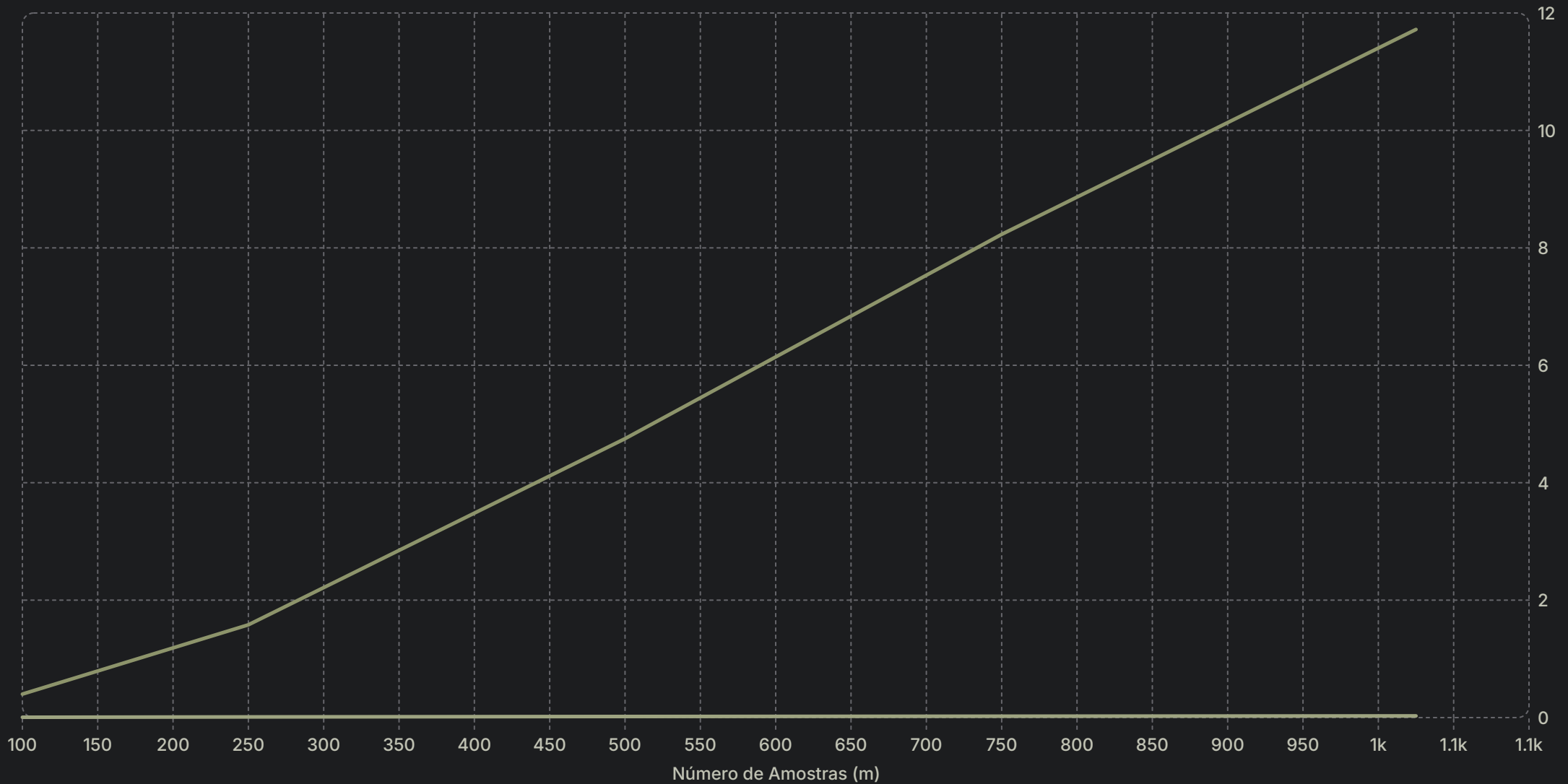
Speedup Final

Ganho de desempenho da versão vetorizada

Análise de Escalabilidade

— Vetorizado

— Iterativo



O ganho de desempenho aumenta exponencialmente conforme o dataset cresce, evidenciando a superioridade da vetorização para Big Data.

Conclusão

Escalabilidade

A vetorização é indispensável para sistemas de IA que operam com Big Data devido ao comportamento linear versus exponencial.

Eficiência

Mesma acurácia e resultado matemático com drástica redução de custo computacional (409,99x mais rápido).

Recomendação

Priorizar o uso de bibliotecas matriciais (NumPy/Pandas) em detrimento de laços explícitos em Python.



Referências Bibliográficas

BISHOP, Christopher M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. *Deep Learning*. Cambridge: MIT Press, 2016.

HARRIS, Charles R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357-362, 2020.

PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design: The Hardware/Software Interface*. 5th ed. San Francisco: Morgan Kaufmann, 2017.

LUIS ANUNCIAÇÃO. *Conceitos e análises estatísticas com R e JASP*.

<https://anovabr.github.io/mqt/index.html#aten%C3%A7%C3%A3o>, 2021